

I. Objectifs

Cet atelier propose une introduction pratique au développement d'un serveur MCP en Python. À travers deux cas concrets, un service d'information aérienne via un fichier json et un service de consultation de livres via une API ouverte.

À la fin de l'atelier, l'étudiant découvrent comment structurer et exposer des données, créer des tools exécutables, définir des ressources MCP et rendre l'ensemble accessible à un client MCP (Claude, Copilot, VSCode Inspector).

II. Travail demandé

1) Mise en place du projet Python :

Créer un nouveau projet ou vous pouvez travailler sur le projet de l'atelier précédent :

Étapes pour créer un nouveau projet :

1. Ouvrir le terminal : **uv venv**
2. Activez l'environnement : **.venv/Scripts/activate**
3. Initialiser un projet uv : **uv init**
4. Installez les dépendances : **uv add "mcp[cli]"**

Cas n°1 : Service d'information aérienne

Créer un script python **flights_server.py** qui expose une ressource et plusieurs tools permettant à un LLM d'accéder à des données métier.

Votre serveur doit :

1. **Contenir une ressource** (resource MCP) représentant un jeu de données stocké dans un fichier JSON (nous allons utiliser le fichier **flights.json**)
2. **Exposer au moins quatre outils (tools)** permettant d'interagir avec ces données :
 - a. un tool pour rechercher un élément par son numéro de vol.
 - b. un tool pour filtrer les données selon un critère choisi (ex : destination)

- c. un tool qui renvoie seulement les éléments ayant un statut particulier (ex : retardé, disponible, réservé...) => **Travail à faire.**
- d. un tool libre basé sur votre propre logique métier => **Travail à faire**

```
from mcp.server.fastmcp import FastMCP
import os
import json

# Chemin absolu vers le dossier resources
FLIGHTS_PATH = os.path.join(os.path.dirname(__file__), "flights.json")

# Création du serveur
mcp = FastMCP(name="Aéroport Info")

def _load_flights():
    with open(FLIGHTS_PATH, "r", encoding="utf-8") as f:
        return json.load(f).get("flights", [])

# -----
# EXPOSITION D'UNE RESSOURCE (fichier JSON lisible par l'LLM)
# -----
@mcp.resource("flights://today")
def flights_resource():
    """
    Resource qui expose la liste des vols du jour.
    L'URL 'flights://today' sera visible par Copilot/Claude.
    """
    with open(FLIGHTS_PATH, "r", encoding="utf-8") as f:
        return f.read()

@mcp.tool()
def find_flight(flight_number: str) -> str:
    """Trouve un vol par son numéro (ex: AF1234)"""

    flights = _load_flights()

    for flight in flights:
        if flight.get("flight_number", "").upper() == flight_number.upper():
            return f"""
✈ Vol {flight["flight_number"]} ({flight["airline"]})
{flight["departure_city"]} → {flight["arrival_city"]}
Départ : {flight["departure_time"]} | Arrivée :
{flight["arrival_time"]}
Statut : {flight["status"]}
            """
```

```

        return f"Vol {flight_number} non trouvé aujourd'hui."

@mcp.tool()
def flights_to(destination: str) -> str:
    """Liste tous les vols à destination d'une ville aujourd'hui."""
    flights = _load_flights()
    matches = [f for f in flights if destination.lower() in
f["arrival_city"].lower()]

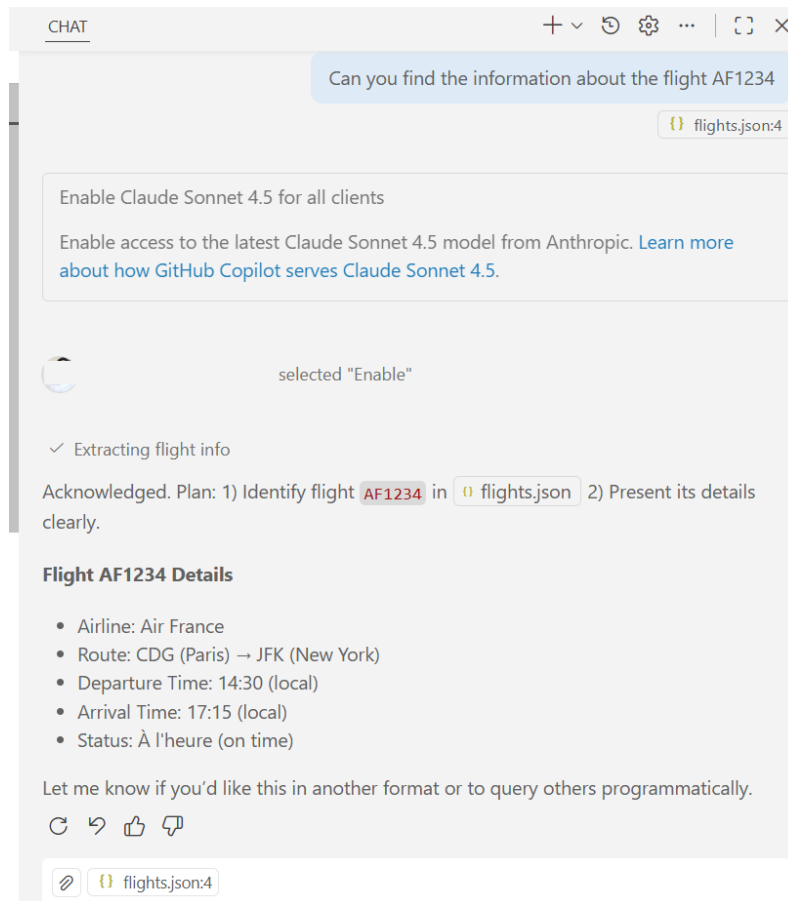
    if not matches:
        return f"Aucun vol trouvé vers {destination.title()} aujourd'hui."

    result = f"Vols vers {destination.title()} ({len(matches)}
trouvé(s)) :\n\n"
    for f in matches:
        result += f"• {f['flight_number']} ({f['airline']}) →
{f['arrival_city']} à {f['arrival_time']} - {f['status']}\n"
    return result.strip()

# Lancement
if __name__ == "__main__":
    mcp.run(transport="stdio") # pour VS Code Copilot

```

3. Tester avec l'inspecteur après, dans Claude Desktop ou Copilot.



Cas n°2 : Service de livres basé sur une API ouverte (<https://openlibrary.org/>)

1. Créer un script python **openlibrary_mcp.py** qui expose deux tools permettant à un LLM d'accéder à des données à partir d'une api externe.

(Vous devez installer **requests** et **fastmcp**)

```
import requests
from fastmcp import FastMCP

BASE_URL = "https://openlibrary.org"

# MCP server global
mcp = FastMCP("OpenLibrary Assistant")

@mcp.tool()
def search_books(query: str) -> list:
    """
    Search for books in OpenLibrary using a query string.
    Returns a list of titles with their work IDs.
    """
```

```

response = requests.get(
    f"{BASE_URL}/search.json",
    params={"q": query},
    timeout=10
)

if response.status_code != 200:
    return {"error": f"API Error {response.status_code}"}

data = response.json()

books = data.get("docs", [])[0:10] # limit to first 10 results

return [
    {
        "title": b.get("title"),
        "author": b.get("author_name", ["Unknown"])[0],
        "year": b.get("first_publish_year"),
        "work_id": b.get("key") # e.g. "/works/OL82563W"
    }
    for b in books
]

@mcp.tool()
def get_book_details(work_id: str) -> dict:
    """
    Retrieve details about a specific book using its work ID.
    Example work_id: /works/OL82563W
    """

    # Normalize input
    if not work_id.startswith("/works/"):
        work_id = f"/works/{work_id}"

    response = requests.get(f"{BASE_URL}{work_id}.json", timeout=10)

    if response.status_code == 404:
        return {"error": "Book not found"}

    if response.status_code != 200:
        return {"error": f"API Error {response.status_code}"}

    return response.json()

```

```
if __name__ == "__main__":  
    mcp.run(transport="stdio")
```

2. Tester avec Claude Desktop ou Copilot.

Recherche un livre du "python"

✓ Optimizing tool selection...

✓ Searching for Python books

I'll query the OpenLibrary MCP to find Python books.

> Ran `search_books` – books (MCP Server) ✓

Plan:

- Show top Python-related book results from OpenLibrary.
- If you want details, I'll fetch any title's info.

Here are 10 popular Python books found:

- Learning Python (Mark Lutz, 1999) — work `"/works/OL2784125W"`
- Python (Mark Lutz, 1998) — work `"/works/OL2784127W"`
- Python Cookbook (Alex Martelli, 2002) — work `"/works/OL14873289W"`
- Fluent Python (Luciano Ramalho, 2015) — work `"/works/OL19932156W"`
- Core Python Programming (R. Nageswara Rao, 2016) — work `"/works/OL30906747W"`
- Programming Python (Mark Lutz, 1996) — work `"/works/OL2784126W"`
- Black Hat Python (Justin Seitz, 2014) — work `"/works/OL19547345W"`
- Think Python (Allen B. Downey, 2009) — work `"/works/OL16976951W"`
- Python (Joshua Welsh, 2017) — work `"/works/OL20212390W"`
- Python For Data Analysis (Wes McKinney, 2012) — work `"/works/OL17422847W"`